

Aula Extra E1 — Análise de Agrupamento: k -Médias

Curso de Aprendizado de Máquina — UFRJ

Leitura recomendada: AME cap. 11; ISLP §12.4

O que você vai aprender

Ao final desta aula você deve ser capaz de:

- distinguir aprendizado supervisionado de *não supervisionado* e dizer por que a avaliação fica mais difícil;
- escolher uma medida de *dissimilaridade* e entender que essa escolha define o que é um “grupo”;
- enunciar o objetivo do k -médias (WCSS) e executar o **algoritmo de Lloyd**;
- explicar por que Lloyd converge, e para o quê;
- escolher K pelo *cotovelo* e pela *silhueta*, sabendo dos limites de ambos;
- descrever o **agrupamento hierárquico** e o papel do *linkage*.

Viramos a chave para o Bloco III: aqui **não há rótulos** Y . O objetivo deixa de ser “prever” e passa a ser “descobrir estrutura”. Isso muda tudo, a começar pela pergunta:

sem uma resposta certa, como sabemos se um agrupamento é bom?

Guarde a pergunta: ela é genuinamente mais difícil do que qualquer coisa do Bloco I, e a resposta honesta — Seção 3.2 — é que não há critério definitivo.

1 Aprendizado não supervisionado

Até aqui observávamos pares $(\mathbf{X}_i, Y_i)$. No **aprendizado não supervisionado** observamos apenas as covariáveis $\mathbf{X}_1, \dots, \mathbf{X}_n$, *sem* rótulos, e buscamos estrutura nos dados. Duas grandes tarefas: *agrupamento* (esta aula) e *redução de dimensionalidade* (Aula E2).

Atenção

Sem rótulos, não há “risco” nem “erro de teste” no sentido das aulas anteriores. Toda a maquinaria das Aulas 01 e 03 — que existia para estimar $\mathbb{E}[L(Y, g(\mathbf{X}))]$ — simplesmente não se aplica, porque não há Y . A avaliação passa a ser mais sutil e frequentemente subjetiva: o agrupamento é “bom” se for *útil* e *interpretável* no problema. Volte a esta caixa ao ler a Seção 3.2.

2 O problema de agrupamento

Agrupar (clustering) é particionar os índices $\{1, \dots, n\}$ em grupos $C_1, \dots, C_K$ disjuntos cuja união é o todo, de modo que elementos do *mesmo* grupo sejam *parecidos* e elementos de grupos *diferentes* sejam *distintos*. “Parecido” exige uma medida de **dissimilaridade** $d(\mathbf{x}_i, \mathbf{x}_j)$. Exemplos:

Medida	$d(\mathbf{x}_i, \mathbf{x}_j)$
Euclidiana	$\sum_{k=1}^d (x_{i,k} - x_{j,k})^2$
Manhattan	$\sum_{k=1}^d x_{i,k} - x_{j,k} $
Cosseno	$1 - \frac{\mathbf{x}_i \cdot \mathbf{x}_j}{\ \mathbf{x}_i\ \ \mathbf{x}_j\ }$
Jaccard	adequada a dados binários (presença/ausência)

Ideia central

A escolha da medida *define* o que significa “grupo”. Um exemplo esclarece: dois textos, um com 50 palavras e outro com 5000, sobre exatamente o mesmo assunto, ficam *longe* na distância euclidiana (as contagens têm magnitudes diferentes) e *perto* na distância cosseno (que só olha a direção, isto é, as proporções relativas de palavras). Por isso o cosseno é padrão em texto (Aula E3), e por isso **escolher a dissimilaridade é a primeira decisão de modelagem** — ela vem antes de qualquer algoritmo.

3 k -Médias

O k -médias usa a distância **euclidiana** e exige fixar K de antemão. Busca a partição que minimiza a **soma de quadrados dentro dos grupos** (WCSS):

$$\min_{C_1, \dots, C_K} \sum_{k=1}^K \frac{1}{|C_k|} \sum_{i,j \in C_k} d^2(\mathbf{x}_i, \mathbf{x}_j) = \min_{C_1, \dots, C_K} \sum_{k=1}^K \sum_{i \in C_k} \|\mathbf{x}_i - \mathbf{c}_k\|^2, \quad (1)$$

onde $\mathbf{c}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$ é o **centroide** do grupo k . A igualdade acima não é óbvia e vale reter: minimizar a dispersão *par-a-par* dentro do grupo equivale a minimizar a soma das distâncias ao *centroide*. É ela que transforma um problema sobre $\binom{|C_k|}{2}$ pares num problema sobre um único ponto por grupo — e é por isso que o algoritmo abaixo é tão simples.

Atenção

Há K^n atribuições possíveis: otimizar (1) exatamente é inviável (o problema é NP-difícil). Usamos uma heurística iterativa — o algoritmo de Lloyd — que encontra um *mínimo local*. É o mesmo tipo de concessão que fizemos ao crescer árvores de forma gananciosa na Aula 06.

3.1 Algoritmo de Lloyd

Algoritmo de Lloyd (k -médias)

1. **Inicialização:** escolha K centroides $\mathbf{c}_1, \dots, \mathbf{c}_K$ (ex. aleatoriamente entre os dados). Itere até convergir:
2. **Atribuição:** associe cada ponto ao centroide mais próximo, $C_k = \{\mathbf{x}_i : \arg \min_r \|\mathbf{x}_i - \mathbf{c}_r\| = k\}$.
3. **Atualização:** recalcule cada centroide como a média do seu grupo, $\mathbf{c}_k \leftarrow \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$.

Proposição 3.1 (Lloyd não aumenta a WCSS). *Cada iteração do algoritmo de Lloyd não aumenta o objetivo (1); como há finitas partições, o algoritmo converge em número finito de passos.*

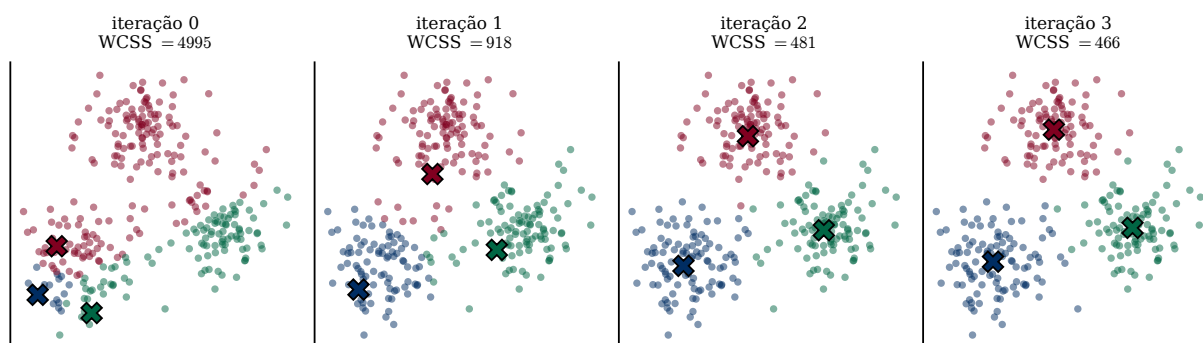


Figura 1: Lloyd em ação sobre 300 pontos, partindo de uma inicialização deliberadamente ruim (os três centroides amontoados no canto inferior esquerdo). Os “X” são os centroides. Em três iterações eles se espalham e encontram os três grupos, e a WCSS cai de 4995 para 918, depois 481 e 466 — decrescendo a cada passo, como a Proposição 3.1 garante, e com ganhos cada vez menores.

Esboço. O passo de *atribuição* reduz (ou mantém) a soma das distâncias ao centroide, pois cada ponto vai ao centroide mais próximo. O passo de *atualização* também reduz (ou mantém), pois a média é o ponto que minimiza $\sum_{i \in C_k} \|\mathbf{x}_i - \mathbf{c}\|^2$. Como o objetivo decresce monotonicamente e só há finitas partições, há convergência.

Atenção

Preste atenção no que a proposição **não** diz. Ela garante que o algoritmo *para*, e que para num ponto onde nenhum dos dois passos melhora nada. Não garante que esse ponto seja o mínimo global de (1) — e em geral não é.

Atenção: Mínimos locais e inicialização

Lloyd converge para um mínimo *local* que **depende da inicialização**: sementes ruins levam a agrupamentos ruins. Soluções:

- rodar várias vezes com inícios diferentes e ficar com a menor WCSS (`n_init` no `scikit-learn`) — barato e eficaz;
- usar *k-médias++*: escolhe os centroides iniciais *espalhados* — o primeiro ao acaso e cada seguinte com probabilidade proporcional ao quadrado da distância ao centroide mais próximo já escolhido, $p_i \propto D^2(\mathbf{x}_i)$. Aumenta muito a chance de bom resultado, e é o padrão do `scikit-learn`.

3.2 Escolhendo o número de grupos K

K é o “*tuning parameter*”, mas — e aqui está a diferença fundamental em relação ao resto do curso — **sem um erro de teste para minimizar**. Não dá para usar validação cruzada, porque não há nada contra o que validar. Restam heurísticas:

- Cotovelo** plote a WCSS contra K . Ela *sempre* cai com K (com $K = n$, $WCSS = 0$), então minimizá-la é inútil; procure o “cotovelo”, onde o ganho marginal despenca.
- Silhueta** mede, para cada ponto, quão melhor ele se encaixa no seu grupo do que no grupo vizinho mais próximo. Ao contrário da WCSS, ela *tem* máximo interior — e por isso pode ser otimizada de fato.

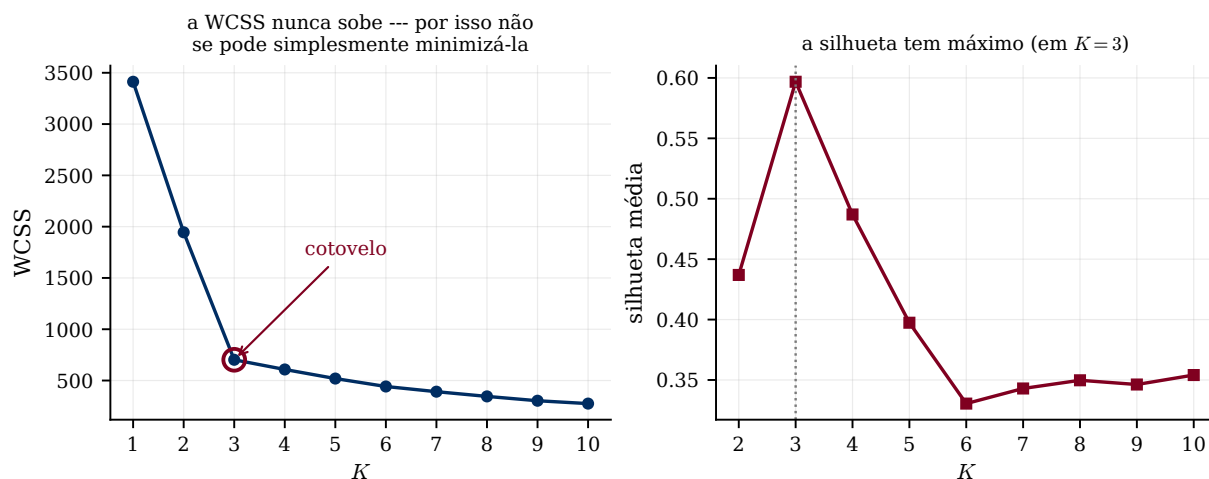


Figura 2: As duas heurísticas nos mesmos dados, que *de fato* têm três grupos. À esquerda a WCSS: ela desaba de $K = 1$ a $K = 3$ e depois cai devagar — o cotovelo em $K = 3$. À direita a silhueta média, que tem um máximo bem definido, também em $K = 3$. Note a assimetria: o cotovelo é um julgamento visual (e nem sempre há um cotovelo), enquanto a silhueta dá um número.

Ideia central

Aqui está a resposta honesta à pergunta de abertura: **não existe critério definitivo**. As duas heurísticas concordaram nesta figura porque os dados foram gerados com três grupos bem separados. Em dados reais elas frequentemente discordam, e nenhum dos dois números tem o *status* que um erro de teste tem. O critério final é o que a caixa lá do começo dizia: o agrupamento é bom se for útil e interpretável no seu problema. Isso não é um defeito da técnica — é o que significa não ter rótulos.

Exemplo 3.2 (Compressão de imagens). Trate cada *pixel* como um ponto em $\mathbb{R}^3$ (cores R, G, B) e rode k -médias com K pequeno. Substituindo cada pixel pela cor do seu centroide, a imagem passa a usar só K cores — uma **compressão** (quantização de cor). K controla a qualidade. (É o exemplo do [AME]; compare com a compressão por SVD da Aula E2, que ataca o mesmo problema por outro caminho.)

4 Agrupamento hierárquico

Ao contrário do k -médias, o agrupamento hierárquico (*aglomerativo*) **não exige fixar K** e produz uma hierarquia de grupos:

1. comece com cada ponto em seu próprio grupo;
2. repetidamente *funda* os dois grupos mais próximos;
3. pare quando restar um único grupo.

O resultado é um **dendrograma** (árvore), que se “corta” a uma altura para obter o número desejado de grupos. A noção de “distância entre grupos” é o *linkage*:

Linkage	Distância entre grupos A e B
Completo (<i>complete</i>)	$\max_{i \in A, j \in B} d(\mathbf{x}_i, \mathbf{x}_j)$ — grupos compactos
Simplex (<i>single</i>)	$\min_{i \in A, j \in B} d(\mathbf{x}_i, \mathbf{x}_j)$ — tende a “encadear”
Médio (<i>average</i>)	média das distâncias entre pares
Ward	minimiza o aumento da WCSS na fusão

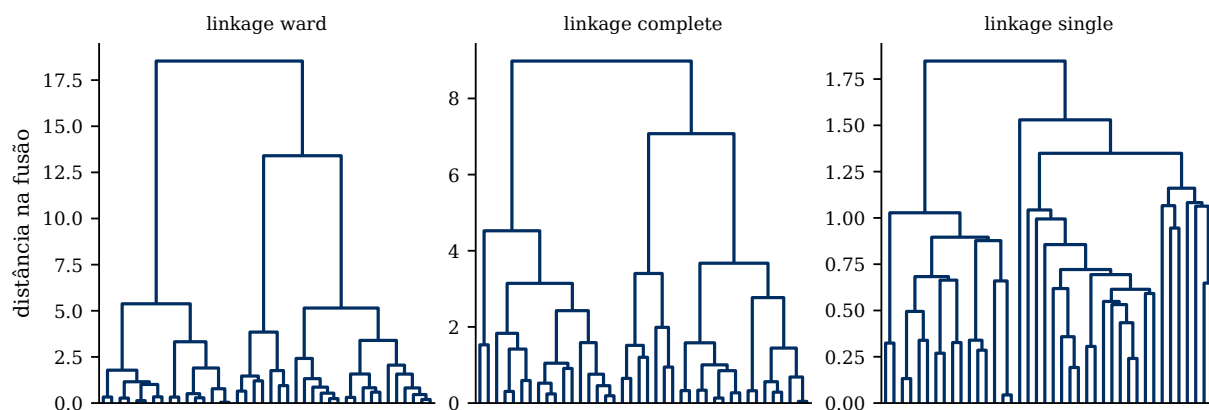


Figura 3: O mesmo conjunto de 40 pontos, três *linkages*. A altura de cada fusão é a distância entre os grupos fundidos, e cortar o dendrograma numa altura dá os grupos correspondentes. Ward e completo produzem árvores equilibradas, com fusões altas e bem separadas — fácil escolher onde cortar. O *single* exibe o *encadeamento* característico: ele funde repetidamente um ponto de cada vez ao mesmo grupo grande, porque basta *um* par próximo para aproximar dois grupos inteiros.

Ideia central

A vantagem do hierárquico é entregar *todas* as partições de uma vez: você decide K depois de olhar a estrutura, em vez de antes. A desvantagem é o custo — $O(n^2)$ de memória só para guardar as distâncias, o que o inviabiliza com centenas de milhares de pontos, onde o k -médias vai bem.

5 Prática em Python

```

1 from sklearn.cluster import KMeans, AgglomerativeClustering
2 from sklearn.metrics import silhouette_score
3 from sklearn.preprocessing import StandardScaler
4
5 X = StandardScaler().fit_transform(X) # PADRONIZE: k-medias usa distancia
6
7 for K in range(2, 11):
8     km = KMeans(n_clusters=K, n_init=10, random_state=0).fit(X)
9     print(K, "WCSS:", km.inertia_, "silhueta:", silhouette_score(X, km.labels_))
10
11 # hierarquico: o dendrograma vem do scipy
12 from scipy.cluster.hierarchy import dendrogram, linkage
13 dendrogram(linkage(X, method="ward"))

```

Atenção

O `StandardScaler` não é opcional aqui, pelo mesmo motivo do KNN (Aula 04): a distância euclidiana soma as coordenadas, e uma variável em unidades grandes domina o agrupamento inteiro. E note que *não há predict* honesto para avaliar — a diferença entre este trecho e todos os anteriores do curso é a ausência de y .

O notebook EXTRA *K-medias (exemplo)* explora o k -médias, a escolha de K e o efeito da inicialização.

Resumo

- Sem rótulos não há risco a estimar: a avaliação é parcialmente subjetiva.
- A *dissimilaridade* escolhida define o que é um grupo — é a primeira decisão de modelagem, não um detalhe.
- *k*-**médias** minimiza a WCSS (1); o problema exato é NP-difícil e usamos o algoritmo de **Lloyd**, que converge (Prop. 3.1) para um mínimo *local* (Fig. 1).
- Inicialização importa: use `n_init` alto e *k*-médias++.
- *K* por *cotovelo* (a WCSS nunca sobe, então não se minimiza) ou por *silhueta* (tem máximo) — e nenhum dos dois é definitivo (Fig. 2).
- **Hierárquico**: dendrograma, sem fixar *K*; o *linkage* muda radicalmente o resultado (Fig. 3); custo $O(n^2)$.
- *Padronize* sempre.

Leitura recomendada. [AME] Capítulo 11: §11.1 (*k*-médias, com o exemplo da compressão de imagens), §11.2 (agrupamento espectral, que não vimos aqui e resolve casos em que o *k*-médias falha por causa da geometria dos grupos) e §11.3 (métodos hierárquicos). [ISLP] §12.4 — §12.4.1 (*k*-médias, com uma figura das iterações de Lloyd equivalente à nossa Figura 1), §12.4.2 (hierárquico e *linkage*) e, muito recomendável, §12.4.3 (*Practical Issues in Clustering*), que discute em detalhe a fragilidade das escolhas desta aula.